SBA-Auth

Adversarial Security Review

REPOSITORY

github.com/SoulboundSecurity/SBA-Auth

AUTHOR

Gakarot

DATE

16 Oct 2026

Summary

Independent review of SBA-Auth's implementation and documented security claims. The report identifies 13 findings (V-01–V-13) and three additional observations, with evidence and limitations noted for each finding.

Review Limitations: This review evaluates the SBA-Auth repository as a standalone artifact, without access to private platform context, deployment-specific assumptions or other repositories. Findings are based on the code, documentation and test scenarios examined during the review; conclusions may differ where additional implementation or deployment context applies.

Findings

V-01 · No service-specific audience binding can allow cross-service signature replay (design + claim)

Evidence	The EIP-712 domain binds signatures to `name`, `version`, `chainId` and `verifyingContract`. The signed structs carry no audience or origin field: `SbaSession` is `{purpose, nonce, validUntil}` and `SbaRequest` is `{purpose, method, path, bodyHash, nonce, issuedAt}` (`client/sba-client.ts`). In an external harness, the same session signature was accepted by two independent servers configured with the same registry, domain name/version, chain and purpose. A request signature was accepted by a second server with a separate nonce cache after the first server accepted it and then denied on a second use at the first. A server configured with a different `purpose` rejected the same session (403), so a custom purpose or domain name mitigates this, but nothing requires one.
Issue	Services that share a compatible SBA-Auth configuration (for example staging and production on one registry) accept each other's signatures. A captured session envelope can be replayed across them for its validity window (default one hour). The domain parameters and signed struct fields do not establish a service-specific audience. A page that can solicit a signature can construct a session payload using a compatible domain and purpose; if the user signs it, the resulting session envelope can be presented to any service using compatible configuration. The nonce does not prevent cross-service replay because the services maintain independent state, while the expiry limits the replay window but does not bind the signature to a particular service. The registry-bound domain identifies a registry deployment; it does not establish which service is meant to accept a signature.
Affected claims	SPEC §3 step 4 ("a signature is bound to one deployment and cannot be obtained for one system and spent on another"); SPEC §6 ("Mitigated by construction: cross-deployment signature phishing (the bound domain)"); README ("domain-bound to one deployment"). USE_CASES §1 contrasts the incumbent stack as "a stored credential or a phishable factor", implying SBA-Auth is not.
Direction	Add a mandatory service-specific `audience` field to both typed-data structs and verify it against the receiving service's configured identity. Alternatively, document that signatures are reusable across services sharing the same domain and purpose configuration.
Limitation	Cross-service replay needs compatible domain and purpose settings, so it is not possible against arbitrary services. Request-mode replay additionally needs the same method, path and body, a signature inside the freshness window, and an unconsumed nonce on the receiving service. The reproduction used an external harness; wallet display behaviour was not tested.

V-02 · Request signature does not cover the query string, and the default path is mount-relative (impl)

Evidence	The signed `SbaRequest` covers `purpose`, `method`, `path`, `bodyHash`, `nonce` and `issuedAt`. The default `pathOf` is `req.path` (`server/sba-auth.ts`), which excludes the query string and is relative to the mount point in Express. In a harness, a signature over `POST /admin/action` was accepted for `/admin/action?to=attacker&amount=1000000` and the handler received `{to:'attacker', amount:'1000000'}`. Under `app.use('/mounted', requestAuth, router)`, a signature over `/mounted/action` was denied (403), while one over `/action` was accepted (200) for the URL `/mounted/action`.
Issue	Any parameter carried in the query string is outside what the signer approves, even when the legitimate client sends it. If a request-mode handler uses `req.query` to choose an operation or its parameters, an attacker who obtains an unused envelope can change them before its first use; the method, path and body checks still pass. Separately, the signed path under a mount point does not identify the full externally addressed route, so the same signed path can refer to different routes under different mounts.

Affected claims	"The signature binds to the exact action" (header comment in `server/sba-auth.ts`) and "what the signer's hardware displays is what executes" (SPEC §4). The docs define the signed action as method, path and body hash and do not mention query strings or mount points.
Direction	Bind the path and canonical query (for example via `originalUrl`) by default, or reject query strings on request-mode endpoints until supported. The client must sign exactly the same canonical path and query the server verifies; changing only the server's `pathOf` is not enough. A `pathOf` override already exists; document its use for mounted routers.
Limitation	Practical impact needs a protected handler that takes sensitive input from the query and a valid unused envelope reaching the server with a modified query before its nonce is consumed (replay protection rejects later uses on the same instance). Impact grows if value-moving endpoints read `req.query`.

V-03 · `deriveIdentity` is optional, so the lying-registry defence depends on configuration (impl vs spec)

Evidence	`verifyIdentity` performs the local identity derivation check only if `config.deriveIdentity` is configured (`server/sba-auth.ts`). In a harness, a predicate returning the admin's identity for any signer, with `deriveIdentity` omitted, allowed an unenrolled stranger's signed session (200). With the same predicate and `deriveIdentity` configured, the request was denied (403).
Issue	If the predicate or RPC returns an incorrect identity matching an authorized admin, the middleware can trust that identity when `deriveIdentity` is absent. The allowlist then authorizes the returned identity without independently confirming that it corresponds to the actual signing address. The claim that a malicious or equivocating registry cannot independently grant access does not hold when `deriveIdentity` is omitted.
Affected claims	SPEC §3 step 6 and §5/§6 describe registry equivocation as "mitigated by construction" and claim that the registry or RPC alone cannot grant access. These protections depend on an optional configuration check rather than an enforced invariant.
Direction	The documented registry-equivocation protection should accurately reflect that it depends on configuring `deriveIdentity`, or the option should be made required.
Limitation	The bypass requires a predicate or RPC response that returns an incorrect identity and a configuration without `deriveIdentity`. The harness used a fake predicate; it does not demonstrate that an honest registry or an otherwise correct RPC will produce this behaviour. The README example configures `deriveIdentity`, but the middleware API does not require it.

V-04 · Request mode signs method, path and a body hash, not the action's content

Evidence	The signed `SbaRequest` struct is `{purpose, method, path, bodyHash, nonce, issuedAt}`, all `string` (defined in `client/sba-client.ts`, used by `server/sba-auth.ts`). Method and path are readable strings; the body appears only as a Keccak-256 hex string (`bodyHashOf`). Application fields inside the body (a recipient or amount, say) are not typed-data fields. SPEC §4 says one action equals "one human-readable confirmation" and that "what the signer's hardware displays is what executes". USE_CASES §1 says each approval is "one hardware-wallet confirmation displaying exactly what is signed" (literally true, but the thing signed is a hash). README and the code comments in `server/sba-auth.ts` repeat "one human-readable" confirmation.
Issue	`bodyHash` binds the request body to the signature, so changes made after signing are detected. But, the typed data does not include structured application fields such as recipient or amount; these values are represented indirectly through `bodyHash`. A compromised or malicious front end can construct a malicious body before requesting the signature, and the resulting signature will still be valid for that body. The protocol therefore does not itself guarantee that the signer can review the actual operation fields in human-readable form. SPEC §4's compromised-browser statement addresses post-signing tampering, but does not prevent a malicious front end from constructing a malicious operation before signing.
Affected claims	SPEC §4, USE_CASES §1/§3, README, `server/sba-auth.ts` header comments. Wallet rendering of SBA's typed data was not tested; this finding concerns the signed schema and the documented guarantee, not what any particular wallet displays.
Direction	Put structured operation fields in the typed data, or sign a summary the server recomputes from the body. Otherwise drop "human-readable" and scope the compromised-browser claim to in-transit tampering.

V-05 · Nonce cache bounds are unvalidated, weakening the single-use signature guarantee (impl)

Evidence	The `nonceCacheMax` configuration is not validated by `assertConfig` (`server/sba-auth.ts`). With `nonceCacheMax: 0`, the same request signature was accepted three times (200, 200, 200). With `nonceCacheMax: -1`, it was also accepted three times (200, 200, 200). With `nonceCacheMax: 3`, a previously accepted signature was accepted again after four other authenticated requests evicted its nonce (200, 200). With `nonceCacheMax: NaN`, replay attempts were rejected (200, 403, 403), but cache eviction was disabled.
Issue	The request-mode single-use guarantee depends on the nonce remaining in the in-memory cache. Zero or negative capacity causes consumed nonces to be removed immediately or not retained, allowing the same signature to be accepted repeatedly. With a positive but limited capacity, authenticated traffic can evict an earlier nonce, allowing its signature to be accepted again while it remains within the freshness window. A `NaN` value prevents eviction and can cause unbounded memory growth.
Affected claims	SPEC §4 describes request-mode signatures as single-use. The implementation does not preserve that guarantee for all accepted `nonceCacheMax` values or after a nonce is evicted from the cache.
Direction	Validate `nonceCacheMax` as a positive integer and make clear that replay protection is bounded by cache capacity and nonce retention.
Limitation	The zero and negative capacity cases require an explicit non-default configuration. The eviction case requires enough authenticated requests to exceed the configured capacity, and the replayed signature must remain within its freshness window. The default capacity is 10,000 entries, so the eviction scenario is less immediate under the default configuration.

V-06 · In-process state makes revocation and replay protection unreliable across restarts and instances (impl)

Evidence	Session revocations are stored in a local `Set`, and consumed request nonces are stored in an in-memory `NonceCache` (`server/sba-auth.ts`). In a harness, a revoked session returned 403 on the original middleware instance but 200 on a fresh instance. A request signature accepted on one instance returned 403 when replayed there, but 200 on a separate instance. The same behaviour applies when process restarts discard the in-memory state.
Issue	Revoking a stolen session does not reliably invalidate it across instances or restarts. If the session remains valid and other checks pass, it can be accepted again where the revocation record is absent. Similarly, a previously consumed request signature can be accepted by another instance or after a restart while its freshness window remains valid. This weakens the guarantees of session revocation and single-use request signatures in deployments where authentication state is not shared. Array-based allowlists are also copied at middleware construction, so changes to the original array do not automatically update existing middleware instances.
Affected claims	SPEC §9 and the internal audit discuss multi-replica deployments backed by shared nonce and revocation state, but the shipped implementation does not expose an injectable shared-store interface. The audit's claim that single-replica deployments get "correct semantics for free" also overlooks the loss of state on process restart.
Direction	The implementation's state-persistence limitation should be explicit: revocation and replay state are local to each middleware instance and do not survive restarts.
Limitation	The harness created independent middleware instances rather than restarting an operating-system process. The restart impact follows from the confirmed use of in-memory state. Replays after a restart require the request signature to remain within its freshness window; session reuse requires the session to remain valid and pass the other authorization checks.

V-07 · Enrollment authority and operators are EOA-only; multisig example is unsupported

Evidence	The contract validates enrollment approvals by recovering an EOA signer with `ecrecover()`, and the server validates authentication signatures with `recoverTypedDataAddress()`; neither path checks ERC-1271 signatures from contract wallets. A harness using a mock ERC-1271 wallet confirmed that the registry rejects its contract-wallet enrollment approval.

Issue	A Safe-style smart-contract wallet cannot directly act as the documented multisig enrollment authority through this signature-verification path. The server's authentication path likewise does not support ERC-1271 signatures from contract-wallet operator addresses. The documented mismatch is the multisig enrollment-authority use case in USE_CASES §1.
Affected claims	USE_CASES §1, which describes the security team's multisig as the enrollment authority.
Direction	Add ERC-1271 support for contract-wallet signatures, or clarify that the implementation supports only EOA-compatible signers.
Limitation	The harness used a mock ERC-1271 wallet, not a deployed Safe. EOA-based MPC/threshold wallets that produce standard ECDSA signatures are not necessarily affected.

V-08 · The on-chain roster records enrollment, not authorization changes (claim)

Evidence	`SoulboundAuthToken.sol` has no burn, revoke or transfer function. `server/sba-auth.ts` requires both an on-chain enrollment and membership in the off-chain allowlist, so granting or removing access emits no on-chain event.
Issue	The roster records enrollment, not current authorization. The off-chain allowlist determines which enrolled identities are currently authorized. Grant and removal history is not recorded on-chain; whether that history is retained elsewhere depends on the deployment. An already-enrolled identity can be added to the allowlist without a new on-chain event. The chain can show when that identity was enrolled, but not when it was granted or removed authorization by the off-chain allowlist. Thus, the registry makes enrollment visible but does not independently evidence every change in who is currently authorized.
Affected claims	SPEC §2 ("a new key means a new, publicly visible enrollment"), README ("can never quietly move to a new key"), USE_CASES §4.
Direction	State that the registry records enrollment while the allowlist controls authorization, and qualify the USE_CASES §4 wording.
Limitation	This is overstated auditability wording, not an exploit. SPEC §2 already says the allowlist is the sole authorizer under open enrollment.

V-09 · Missing raw-body capture can bypass request-body binding for an empty-body signature (impl)

Evidence	`rawBodyOf` in `server/sba-auth.ts` falls back to an empty string when raw-body capture is missing. In a harness without raw-body capture, a signature over an empty body was accepted (200) when the request contained `{"action":"drain"}`, and the handler received that attacker-supplied body. A signature over a non-empty body with raw-body capture missing was rejected (403).
Issue	If an application fails to configure raw-body capture, the middleware can treat the missing body as an empty string instead of rejecting the request. An attacker with a valid, unused signature over an empty body can therefore submit a different body that the handler processes, even though that body was not covered by the signature. This breaks the request-mode guarantee that the executed body matches the signed body.
Affected claims	SPEC §4 states that the signature covers the exact raw body bytes. The internal audits dated 2026-10-11 and 2026-10-13 also claim that missing raw-body capture fails closed, which the reproduced empty-body case contradicts.
Direction	Missing raw-body capture should result in rejection rather than being treated as an empty body.
Limitation	Exploitation requires an integration that omits raw-body capture, a valid unused signature over an empty body, and a protected handler that processes the submitted body. The harness confirms this conditional behavior; it does not establish that the repository's example server is vulnerable when configured as documented.

V-10 · Dual-control pattern is under-specified and fails if read as a middleware chain (claim)

Evidence	USE_CASES §3 says: "Dual-control is two request-mode gates in sequence with disjoint allowlists." The middleware reads one fixed header (`x-sba-request`, hard-coded via `SBA_DEFAULTS`, not configurable)

	carrying one signer's envelope. In a harness with gate A (signer A) followed by gate B (signer B) on the same route, A's envelope passed gate A and failed gate B (403), and B's envelope failed gate A (403).
Issue	Chained on one route, two gates with disjoint allowlists can never both pass, because one request carries one signer. The sentence can only work as a multi-request approval workflow (each approver signs a separate request), which needs application state linking the approvals (a pending action, a matching body hash, an expiry) that the repo neither provides nor describes.
Affected claims	USE_CASES §3.
Direction	Describe dual-control as an application-level two-step approval workflow and say what state it needs, or add multi-envelope support.
Limitation	Dual control is achievable with a separate workflow, and the sentence does not say which reading is meant. This concerns the documented pattern's fit with the shipped middleware, not a security flaw.

V-11 · Attributable audit trail depends on adopter-side evidence retention (claim)

Evidence	README describes per-action confirmation as doubling as an attributable audit trail. USE_CASES §3 claims that the signed envelope and enrollment can establish which enrolled identity approved exactly which bytes and when. But, the middleware in `server/sba-auth.ts` does not persist approval records or expose an audit callback for successful or failed authentication. It attaches authentication details to `req.sbaAuth`, leaving any logging and retention to the integrating application. The library does not automatically preserve the signed envelope and request body as an audit record.
Issue	A verifiable audit record is possible only if the integrating application retains the signed envelope, exact request body, and relevant enrollment evidence. The middleware does not persist those materials itself, so the library does not by itself guarantee a durable audit trail. Additionally, `issuedAt` is supplied by the signer; it proves the timestamp included in the signed payload, not an independently attested time of approval.
Affected claims	README's claim that per-action confirmation doubles as an attributable audit trail, and USE_CASES §3's claim that the envelope and enrollment establish which identity approved exactly which bytes and when.
Direction	Clarify that creating a verifiable audit trail requires the integrating application to retain the signed envelope, exact request body, and relevant enrollment evidence. Avoid implying that SBA-Auth itself provides durable logging or an independently attested approval time.
Limitation	This finding does not establish that applications using SBA-Auth cannot maintain an audit trail. Integrators can implement their own logging and retention. The issue is the gap between the library's claims and its built-in audit functionality, not an authentication bypass.

V-12 · Asymmetric clock-skew tolerance versus SPEC §9 (spec mismatch)

Evidence	With the default 90-second request freshness window, a timestamp 80 seconds in the past was accepted (200), while one 100 seconds in the past was rejected (403). A timestamp 50 seconds in the future was accepted (200), while one 70 seconds in the future was rejected (403). In `server/sba-auth.ts`, the past bound uses `maxAgeMs` without additional tolerance, while the future bound allows 60 seconds of clock skew.
Issue	The implementation applies different time allowances to past and future timestamps. This differs from SPEC §9, which states that both freshness bounds have 60 seconds of clock-skew tolerance. As a result, a request may be rejected even when its timestamp falls within the tolerance described by the specification. This is a specification mismatch, not a demonstrated authentication bypass.
Affected claims	SPEC §9 states that clock skew is handled with 60 seconds of tolerance on both freshness bounds. The implementation applies future-clock tolerance but does not apply corresponding tolerance to the past-age bound.
Direction	Document the actual timestamp acceptance rules or align the implementation with the two-sided tolerance described in SPEC §9.
Limitation	The harness confirms the acceptance boundaries for the tested timestamps and default configuration. Practical impact depends on clock differences and the configured freshness window. No authentication bypass was demonstrated.

V-13 · SPEC misstates the verification sequence for body tampering

Evidence	SPEC §4 states that body tampering “dies at step 3”, but SPEC §3 step 3 is the freshness check. In `server/sba-auth.ts`, request mode checks the signed method, path and body hash before validating `issuedAt` freshness. These request-binding checks are not separately listed in the seven-step gauntlet. The documented sequence also omits the session revocation check and request nonce consumption, and does not accurately represent the mode-specific execution order.
Issue	The SPEC's description of the verification sequence is inaccurate. A reader following the documented gauntlet cannot determine the complete set and actual order of checks performed by the middleware. This is a documentation accuracy issue; the finding does not establish a failure of the implemented body-hash verification or an authorization bypass.
Affected claims	`SPEC.md` §3 (verification gauntlet) and §4 (body-tampering explanation).
Direction	Update the gauntlet to reflect the implementation's actual verification order, explicitly document request-binding checks, session revocation and request nonce consumption, and correct the reference to step 3. Keep the session-mode and request-mode flows distinct where their checks differ.

Notes

- **CLA.** `CLA.md` refers to "SBF Protocol" rather than SBA-Auth, and README says contributions are accepted under it.
- **License identifiers.** Source headers use `AGPL-3.0`, `package.json` uses `AGPL-3.0-only`, and `LICENSE` grants "either version 3 of the License, or (at your option) any later version." The dual-licence model depends on the grant, so these are worth aligning.
- **Tests and CI.** The repository has no unit tests, Solidity tests or CI workflow. `example/demo.ts` runs 8 checks, while the 2026-10-11 audit says the rejection matrix "demonstrates each guarantee executably." Cases not exercised include wrong chain or `verifyingContract`, wrong purpose, expired session, stale or future `issuedAt`, method or path mismatch, an enrolled but not allowlisted key, and a lying predicate.